Java 基础 / 集合 & Go 面试八股笔记

面向 Go 后端 / AI Agent 开发实习

第一部分 Java 基础

一、Java 概述

二、数据类型

三、面向对象

四、常用类与关键方法

五、泛型、反射与注解

六、异常

七、Java 新特性、序列化与 IO

第二部分 Java 集合

八、集合概览

九、List

十、Set

十一、Map

十二、Queue

第三部分 Go

十三、Go 基础

十四、slice 与 map

十五、channel 与 select

十六、sync 与并发原语

十七、context

十八、interface 与反射

十九、GMP 调度模型

二十、内存管理与 GC

二十一、常见并发模式与代码题

> 使用说明：按「主题 → 知识点」组织的结构化笔记，知识框架主要参考小林coding 面试题合集（Java 基础、Java 集合、Go），已重新整理改写。Java 部分按要求只保留**基础与集合**（不含 JVM、JUC 并发、Spring）。先记加粗关键词，再展开。涉及版本差异处已标注版本号。

第一部分 Java 基础

一、Java 概述

1. Java 特点与跨平台原理

- 特点：**平台无关**（一次编译，到处运行）、**面向对象**、**自动内存管理（GC）**、生态成熟（Spring 等）、内置多线程支持。
- 劣势：相对 C++/Rust/Go 启动慢、内存占用大，语法样板代码较多。
- **跨平台原理**：`javac` 把源码编译为平台无关的**字节码（.class）**，由各平台对应的 **JVM** 翻译成本地机器码执行。**跨平台的是字节码，JVM 本身不跨平台**（不同系统安装不同 JVM）。
- JVM 上不止能跑 Java，Kotlin、Scala 等编译为字节码同样可运行。

2. JDK、JRE、JVM

- **JVM**：运行字节码的虚拟机，负责解释/编译执行、内存管理、GC。
- **JRE** = JVM + 核心类库，运行 Java 程序的最小环境。
- **JDK** = JRE + 开发工具（`javac`、`jdb`、`jar`、`javap` 等）。关系：**JDK ⊃ JRE ⊃ JVM**（JDK 11 起官方不再单独提供 JRE 安装包）。

3. 编译与解释并存

- 编译期：`javac` 生成字节码。
- 运行期：**解释器**逐条解释执行；JVM 用方法调用计数器、回边计数器识别**热点代码**，由 **JIT 即时编译器**编译为本地机器码缓存，后续直接执行。
- 因此 Java 是**编译 + 解释混合**模式；C/C++ 是典型编译型，Python/JS 是典型解释型。

4. 值传递

- **Java 只有值传递**。
- 基本类型：传递值的副本，方法内修改不影响原变量。
- 引用类型：传递**引用（地址）的副本**——通过副本修改对象内部字段会影响原对象；但让副本指向新对象，不影响原引用。

二、数据类型

5. 八种基本数据类型

类型	字节	范围 / 说明	默认值	包装类
byte	1	-128 ~ 127	0	Byte
short	2	-2^15 ~ 2^15-1	0	Short
int	4	-2^31 ~ 2^31-1	0	Integer
long	8	-2^63 ~ 2^63-1，字面量加 L	0L	Long
float	4	单精度，字面量加 f	0.0f	Float
double	8	双精度，小数默认类型	0.0d	Double
char	2	无符号，0 ~ 65535（UTF-16 码元）	'000'	Character
boolean	未明确规定	true / false	false	Boolean

- 整数字面量默认 int，小数默认 double。

6. 类型转换与常见问题

- **自动（隐式）转换**：小范围 → 大范围（int → long，float → double），安全。
- **强制转换**：大范围 → 小范围，可能**溢出**（如 `(byte)300 == 44`，保留低 8 位）或**精度丢失**（`(int)3.14 == 3`）。
- long ↔ int：int → long 安全；long → int 需强转，超出范围时截断高位。
- **引用类型转换**：向上转型自动且安全；向下转型需强转，实际类型不符抛 `ClassCastException`，先用 `instanceof` 判断（Java 16 起支持 `if (obj instanceof Dog d)` 模式匹配）。
- 字符串转换：`Integer.parseInt()`、`Double.parseDouble()`、`String.valueOf()`。

7. 包装类、装箱拆箱与 Integer 缓存

- **装箱** `Integer i = 10;` 编译为 `Integer.valueOf(10)`；**拆箱** `int n = i;` 编译为 `i.intValue()`。发生在赋值和方法调用时。
- **为什么需要 Integer**：泛型/集合只能存引用类型（`List<Integer>`）；可用 `null` 表示「无值」；提供 `parseInt`、`toHexString` 等工具方法。
- **为什么保留 int**：基本类型直接存值，读写效率高；64 位 JVM 开启压缩指针时一个 Integer 对象约 16 字节，int 仅 4 字节。
- **IntegerCache**：`valueOf` 缓存 **-128 ~ 127**（上限可通过 JVM 参数调大），范围内复用同一对象。
  - `Integer a=127, b=127; a==b` → true；`128 == 128`（两个 Integer）→ false（默认缓存范围下）。

- **Integer 比较数值用 `equals`**（兼容 null 用 `Objects.equals`）；Integer 与 int 用 `==` 比较会先拆箱。

- 坑：**null 拆箱抛 NPE**；循环中 `Integer sum += i` 会反复装箱创建大量对象；`new Integer()` 自 JDK 9 起废弃。

8. BigDecimal 与浮点精度

- double 是**二进制浮点数**，无法精确表示 0.1 等十进制小数：`0.1 + 0.2 = 0.30000000000000004`。
- **金额等精确计算用 BigDecimal**（任意精度整数 + scale）。
- 注意：
  - 用**字符串构造** `new BigDecimal("0.1")` 或 `BigDecimal.valueOf(0.1)`，不要 `new BigDecimal(0.1)`；
  - **除法要指定精度和舍入模式**（如 `RoundingMode.HALF_UP`），否则除不尽抛 `ArithmeticException`；
  - **比较数值用 `compareTo`**，`equals` 会同时比较 scale（`1.0` 与 `1.00` 不 equals）；
  - BigDecimal 不可变，运算结果要接住返回值。

三、面向对象

9. 封装、继承、多态

- **封装**：属性和行为结合，隐藏内部细节，通过公开接口交互（private 字段 + getter/setter）。
- **继承**：子类复用父类属性和方法，Java **类单继承、接口多实现**。
- **多态**：同一接口不同实现。
  - **编译时多态**：方法重载；
  - **运行时多态**：方法重写 + 父类引用指向子类对象，运行时按**实际类型**动态分派。
- 多态的意义：提高扩展性，是策略模式、依赖倒置、消除 if-else 等设计的基础。

10. 重载与重写

对比	重载 Overload	重写 Override
位置	同一类（或继承得到的方法）	子类与父类之间
方法签名	方法名同，**参数列表不同**	方法名、参数列表**相同**
返回值	无要求（**不能仅靠返回值区分**）	相同或为其子类型（协变返回）
访问权限	无要求	**不能比父类更严格**
异常	无要求	不能抛出新的或更宽的受检异常
确定时机	**编译期**	**运行期**

- `final` 方法不能重写；`private` 方法不被继承；`static` 方法同名属于**隐藏**而非重写。建议加 `@Override` 让编译器检查。

11. 抽象类与接口

- **抽象类**：`abstract` 修饰，不能实例化（可有构造器，供子类调用）；可有成员变量、构造方法、具体方法和抽象方法；单继承；不能用 `final` 修饰（互斥）。
- **接口**：定义行为规范，可多实现；成员变量默认 `public static final`；不能有构造方法。
  - 抽象方法默认 `public abstract`；
  - **Java 8**：`default` 默认方法、`static` 静态方法；
  - **Java 9**：`private` 私有方法（辅助 default 方法）。
- 选择：表达「**is-a**」、需要共享状态和代码 → 抽象类；表达「**能力 / can-do**」、需要多实现 → 接口。

12. 面向对象设计原则（SOLID + 迪米特）

- **单一职责 SRP**：一个类只负责一项职责。
- **开闭原则 OCP**：对扩展开放，对修改关闭。
- **里氏替换 LSP**：子类对象能替换父类对象而不改变程序正确性。
- **接口隔离 ISP**：接口小而专，客户端不依赖不需要的方法。
- **依赖倒置 DIP**：高层和低层都依赖抽象，面向接口编程。
- **迪米特法则**：只与直接朋友通信，减少耦合。

13. 内部类、static 与 final

- **非静态内部类**：持有外部类实例引用（编译器在构造方法中传入外部类对象），可直接访问外部类所有成员；**可能导致外部类无法回收（内存泄漏）**。
- **静态内部类**：不依赖外部类实例，只能直接访问外部类静态成员（单例的静态内部类写法即利用其延迟加载）。
- **static**：修饰变量（类变量，所有实例共享一份）、方法（不能直接访问实例成员，不能被重写只能被隐藏）、代码块（类初始化时执行一次，先于构造方法）、内部类。
- **final**：修饰**类**不能被继承（如 String）；修饰**方法**不能被重写；修饰**变量**只能赋值一次——基本类型值不变，引用类型**引用不变但对象内容可变**。

14. 深拷贝与浅拷贝、对象创建方式

- **浅拷贝**：复制对象本身，引用类型字段只复制引用，新旧对象共享内部对象；`Object.clone()` 默认浅拷贝。
- **深拷贝**：递归复制所有引用对象。实现：重写 `clone()` 逐层克隆；**序列化 + 反序列化**；手动递归复制（或 JSON 工具转换）。

- **创建对象的方式**：`new`；**反射**（`Constructor.newInstance()`，`Class.newInstance()` JDK 9 起废弃）；`clone()`（不调用构造器）；**反序列化**（不调用构造器）；工厂方法（如 `Integer.valueOf`）。

四、常用类与关键方法

15. String 不可变与字符串常量池

- **不可变原因**：类为 `final`，内部存储数组 `private final`（JDK 9 起由 `char[]` 改为 `byte[]` + coder，节省 Latin-1 字符空间），且不对外提供修改方法。
- 不可变的好处：**线程安全**、**可缓存 hashCode**（适合做 HashMap key）、支持**字符串常量池**复用、安全（参数不被篡改）。
- 常量池：字面量 `"abc"` 放入字符串常量池（JDK 7 起位于堆中）复用；`new String("abc")` 会在堆上新建对象（若池中没有 “abc” 还会在池中创建）。
- 编译期常量折叠：`"a" + "b"` 编译时合并为 `"ab"`；`intern()` 返回池中引用。

16. String、StringBuilder、StringBuffer

特性	String	StringBuilder	StringBuffer
可变性	不可变	可变	可变
线程安全	不可变，天然可共享	不安全	方法加 `synchronized`，安全
场景	少量拼接、存储传递	**单线程频繁拼接（首选）**	多线程共享同一实例

- 循环内用 `+` 拼接会反复创建对象，应改用 StringBuilder。
- StringBuffer 单个方法线程安全，**「先判断再追加」等复合操作仍需额外同步**。

17. == 与 equals、hashCode

- `==`：基本类型比较**值**；引用类型比较**是否同一对象（地址）**。
- `equals`：Object 默认实现等价于 `==`；String、Integer 等重写为**比较内容**。
- **equals 与 hashCode 约定**：
  - equals 为 true → hashCode **必须相等**；
  - hashCode 相等 → equals 不一定为 true（哈希冲突）；
  - **重写 equals 必须重写 hashCode**，否则 HashMap/HashSet 中「业务相等」的两个对象落到不同桶，导致查不到或去重失败。
- 作为 HashMap key 后**不要修改参与 hashCode 计算的字段**（建议用不可变字段）。

18. Object 类的方法

- `equals()`、`hashCode()`、`toString()`（默认 `类名@哈希码十六进制`）。
- `getClass()`：返回运行时类，`final` 不可重写。

- `clone()`：`protected`，需实现 `Cloneable`，否则抛 `CloneNotSupportedException`；默认浅拷贝。
- `wait()`（3 个重载）、`notify()`、`notifyAll()`：必须在 `synchronized` 块中调用，否则抛 `IllegalMonitorStateException`；wait 会**释放锁**。
- `finalize()`：JDK 9 起废弃、JDK 18 起标记为待移除，执行时机不确定，资源释放应使用 try-with-resources。

五、泛型、反射与注解

19. 泛型与类型擦除

- 作用：**编译期类型检查**（避免运行时 `ClassCastException`）、**消除强制转换**、**代码复用**。
- **类型擦除**：编译后类型参数替换为上界（无上界则 Object），必要处插入强转；`List<String>` 和 `List<Integer>` 运行时是同一个 Class。
  - 类、字段、方法声明上的泛型信息仍保存在 class 文件 Signature 属性中，可通过反射读取。
- 限制：不能用基本类型（`List<int>` 不合法）；不能 `new T()`；不能创建泛型数组；静态成员不能使用类的类型参数。
- **通配符**：`List<Integer>` 不是 `List<Number>` 的子类型。
  - `? extends T`：上界，适合**读取（生产者）**；
  - `? super T`：下界，适合**写入（消费者）**。即 **PECS**：Producer Extends, Consumer Super。

20. 反射

- 定义：**运行时**获取类的结构信息（构造器、方法、字段），并动态创建对象、调用方法、访问字段。
- 核心类：`Class`（获取方式：`类名.class`、`obj.getClass()`、`Class.forName("全限定名")`）、`Constructor`、`Method`（`invoke`）、`Field`。
- `getMethod` 获取 public 方法（含继承）；`getDeclaredMethod` 获取本类声明的所有方法（含 private，不含继承）；访问私有成员需 `setAccessible(true)`，受 Java 9 模块系统封装限制。
- 应用：Spring IoC 创建 Bean 与依赖注入、**动态代理**（AOP）、JDBC `Class.forName` 加载驱动、ORM 字段映射、JSON 序列化、注解解析。
- 缺点：绕过编译期检查，错误延迟到运行时；有性能开销（可缓存 Method 对象）；破坏封装。

21. 注解

- 本质：继承 `java.lang.annotation.Annotation` 的**特殊接口**；运行时通过反射获取到的是 JDK 生成的**动态代理对象**，属性值通过 `AnnotationInvocationHandler` 从 `memberValues` Map 中取得。

- **元注解**：
  - `@Retention`：`SOURCE`（仅源码，如 `@Override`）、`CLASS`（保留到 class 文件，默认）、`RUNTIME`（运行时可反射读取）；
  - `@Target`：作用位置（TYPE、FIELD、METHOD、PARAMETER、CONSTRUCTOR 等）；
  - `@Documented`、`@Inherited`、`@Repeatable`（Java 8）。
- 解析方式：编译期（注解处理器 APT，如 Lombok）或运行期反射（`getAnnotation`、`isAnnotationPresent`）。

六、异常

22. 异常体系

- 根类 `Throwable`：
  - **Error**：严重错误，程序通常无法处理，如 `OutOfMemoryError`、`StackOverflowError`；
  - **Exception**：
    - **受检异常（Checked）**：编译期必须 `try-catch` 或 `throws` 声明，如 `IOException`、`ClassNotFoundException`；
    - **非受检异常（RuntimeException 及子类）**：程序逻辑错误导致，编译器不强制处理，如 `NullPointerException`、`ArrayIndexOutOfBoundsException`、`ClassCastException`、`IllegalArgumentException`。
- 非受检异常 = RuntimeException + Error。

23. 异常处理机制

- `try-catch`：多个 catch 时**子类异常在前、父类在后**；无匹配则向调用方传播。
- `throw` 在方法体内抛出异常对象；`throws` 在方法签名上声明。
- `finally`：通常一定执行（JVM 退出 `System.exit`、进程被杀等情况除外）。
  - **finally 中的 return 会覆盖 try 中的 return**（`try{return "a"} finally{return "b"}` 返回 “b”），还会吞掉异常，**不要在 finally 中 return**；
  - 若 finally 只修改变量而不 return，try 中 return 的基本类型值已暂存，不受影响。
- **try-with-resources**（Java 7）：资源实现 `AutoCloseable` 后自动关闭，按声明**逆序**关闭；close 抛出的异常作为 suppressed 异常附加到主异常。
- 规范：不要空 catch；包装异常时保留 cause；能处理就处理，不能处理就上抛。

七、Java 新特性、序列化与 IO

24. Java 8 新特性

- **Lambda 表达式**：`(参数) -> 表达式`，简化**函数式接口**（仅一个抽象方法，`@FunctionalInterface`，如 `Runnable`、`Comparator`、`Function`、`Predicate`、`Supplier`、`Consumer`）的实现。
- **方法引用**：`System.out::println`、`String::length`。
- **Stream API**：链式处理集合，**中间操作**（`filter`、`map`、`sorted`、`distinct`，惰性求值）+ **终端操作**（`collect`、`forEach`、`count`、`reduce`）；`parallelStream()` 基于 ForkJoinPool 公共池，适合 CPU 密集且无共享状态的任务。
- **Optional**：包装可能为 null 的值，`ofNullable`、`orElse`、`map`，减少 NPE。
- 接口 default / static 方法；新日期时间 API（`LocalDateTime`，不可变、线程安全）；`CompletableFuture` 异步编排。

25. Java 17 / 21 常见新特性

- **Java 17（LTS）**：`record` 记录类（自动生成构造器、equals/hashCode/toString，Java 16 正式）、`sealed` 密封类（限制可继承的子类，Java 17 正式）、文本块 `"""`（Java 15 正式）、`switch` 表达式（Java 14 正式）。
- **Java 21（LTS）**：**虚拟线程**（JEP 444，轻量级用户态线程，适合大量阻塞 IO 任务）、**switch 模式匹配**（JEP 441）、**记录模式**（JEP 440，解构 record）、有序集合 `SequencedCollection`（JEP 431）。

26. 序列化

- 实现 `Serializable`（标记接口）或 `Externalizable`；通过 `ObjectOutputStream.writeObject` / `ObjectInputStream.readObject` 完成。
- **serialVersionUID**：版本号，反序列化时校验类版本是否一致；未显式声明时由编译器根据类结构计算，类修改后会变化导致 `InvalidClassException`，建议显式声明。
- `transient` 和 `static` 字段不参与默认序列化。
- 反序列化**不调用**类的构造器；Java 原生序列化存在安全风险和性能问题，跨语言常用 **JSON、Protobuf**、Hessian 等。
- 自定义序列化协议要点：魔数 + 版本 + 长度 + 类型的消息头；字段编号便于兼容演进；处理 TCP 粘包拆包；限制长度防恶意输入。

27. BIO、NIO、AIO

- **BIO**（`java.io`）：同步阻塞，一个连接一个线程，连接多时线程资源耗尽。
- **NIO**（Java 1.4，`java.nio`）：同步非阻塞 + **IO 多路复用**，三大组件：

    - **Channel** 通道（双向，可设为非阻塞）；
    - **Buffer** 缓冲区（position / limit / capacity，读写切换用 `flip()`）；
    - **Selector** 多路复用器：一个线程监听多个 Channel 的就绪事件（ACCEPT、READ、WRITE），Linux 底层通常基于 epoll。
    - 「同步」指就绪后仍需应用线程自己调用 read/write 搬运数据。
- **AIO**（Java 7，NIO.2）：异步非阻塞，操作完成后通过回调/Future 通知。
- **Netty**：基于 NIO 的 Reactor 网络框架，bossGroup 负责 accept，workerGroup 负责读写；ChannelPipeline + ChannelHandler 处理编解码与业务；EventLoop 中不要执行阻塞操作。

第二部分 Java 集合

八、集合概览

28. 集合体系

- **Collection**（单列）：
  - **List**（有序、可重复）：`ArrayList`、`LinkedList`、`Vector`、`CopyOnWriteArrayList`；
  - **Set**（不可重复）：`HashSet`、`LinkedHashSet`（插入顺序）、`TreeSet`（排序）；
  - **Queue**：`ArrayDeque`、`PriorityQueue`、`LinkedList`、各种 `BlockingQueue`。
- **Map**（键值对）：`HashMap`、`LinkedHashMap`、`TreeMap`、`Hashtable`、`ConcurrentHashMap`。
- **数组 vs 集合**：数组长度固定、可存基本类型；集合长度可变、只能存引用类型、提供丰富操作。
- **Collection vs Collections**：前者是集合接口；后者是工具类（`sort`、`reverse`、`synchronizedList`、`unmodifiableList`）。

29. 线程安全的集合

- 早期同步容器：`Vector`、`Hashtable`、`Collections.synchronizedXxx()`（方法级 synchronized，粒度粗）。
- `java.util.concurrent` 并发容器：`ConcurrentHashMap`、`CopyOnWriteArrayList` / `CopyOnWriteArraySet`、`ConcurrentLinkedQueue`、`ConcurrentSkipListMap`、各种 `BlockingQueue`。

30. fail-fast 与 fail-safe

- **fail-fast**：`ArrayList`、`HashMap` 等迭代时检查 `modCount` 与 `expectedModCount`，遍历期间被结构性修改（非迭代器自身的增删）立即抛 `ConcurrentModificationException`。**只是尽力检测，不能作为线程安全保证**。
- **fail-safe / 弱一致**：并发容器（`CopyOnWriteArrayList` 遍历快照，`ConcurrentHashMap` 弱一致迭代器）遍历时不抛异常，但**不一定看到最新数据**。
- **不可修改 vs 不可变**：`Collections.unmodifiableList(list)` 只是只读视图，原集合修改后视图可见变化；`List.of()`（Java 9）是真正的不可变集合（且不允许 null 元素）。

九、List

31. ArrayList 与 LinkedList

对比	ArrayList	LinkedList
底层	**动态数组** `Object[]`	**双向链表**
随机访问	O(1)，实现 `RandomAccess`	O(n)
头部/中间插入删除	O(n)，需移动元素	定位 O(n)，修改指针 O(1)
尾部插入	均摊 O(1)	O(1)
内存	连续、紧凑，可能有预留空间；缓存友好	每个节点额外存 prev/next 指针

- 两者**都不是线程安全的**；实际开发中绝大多数场景用 ArrayList。LinkedList 还实现了 `Deque`，可作队列/栈（但通常 `ArrayDeque` 更快）。
- **ArrayList vs Vector**：Vector 方法加 synchronized、默认 2 倍扩容，已基本淘汰。

32. ArrayList 扩容机制

- 无参构造时为空数组，**首次 add 时才分配默认容量 10**（懒加载）。
- 容量不足时扩容为原来的 **1.5 倍**（`oldCapacity + (oldCapacity >> 1)`），若仍不够则直接用所需最小容量；通过 `Arrays.copyOf` 复制到新数组。
- 优化：预知元素数量时构造指定容量，或调用 `ensureCapacity`，避免多次扩容。

33. ArrayList 线程不安全的表现与解决

- 不安全原因：`elementData[size++] = e` 不是原子操作，并发下可能出现：
  - **元素覆盖 / 丢失**（两个线程拿到相同 size 写入同一位置）；
  - **数组越界**（都判断容量够，扩容前同时写入）；
  - size 与实际元素不一致、出现 null。
- 解决：`Collections.synchronizedList()`、`Vector`、**`CopyOnWriteArrayList`**（读多写少）、或外部加锁。
- **CopyOnWriteArrayList**：**写时复制**——写操作加锁（JDK 8 为 ReentrantLock，JDK 9+ 为 synchronized），复制一份新数组修改后替换 volatile 数组引用；读操作**不加锁**读旧数组。适合**读多写少**，缺点是写开销大、内存翻倍、读到的可能不是最新数据。

34. List 使用细节

- **遍历时删除**：普通 for-each 中 `list.remove()` 会触发 `ConcurrentModificationException`；应使用 **`Iterator.remove()`**、`removeIf()`（Java 8），或倒序下标 for 循环删除。
- `remove(1)`：`List<Integer>` 调用 `remove(int index)` **按下标删除**；按元素删除需 `remove(Integer.valueOf(1))`。

- **subList** 返回原列表的**视图**而非副本，修改会互相影响；原列表结构修改后再操作 subList 会抛 CME。
- **数组与 List 互转**：`Arrays.asList(arr)` 返回固定大小的视图（不支持 add/remove，且基本类型数组会被当成一个元素）；可 `new ArrayList<>(Arrays.asList(arr))`；List 转数组用 `list.toArray(new String[0])`。

十、Set

35. HashSet 去重原理

- **底层是 HashMap**：元素作为 key，value 是共享的占位对象 `PRESENT`。
- 去重依赖 key 的 **hashCode() + equals()**：先按 hash 定位桶，再用 equals 判断是否已存在。
- **自定义对象去重失败**常见原因：只重写了 equals 没重写 hashCode；或对象存入后修改了参与哈希计算的字段。

36. LinkedHashSet、TreeSet 与 Set 排序

- **LinkedHashSet**：底层 LinkedHashMap，维护**插入顺序**。
- **TreeSet**：底层 **TreeMap（红黑树）**，按自然顺序（`Comparable`）或 `Comparator` **排序**，增删查 O(log n)。
- **TreeSet 去重依据是 `compareTo` / `compare` 返回 0**，而不是 equals：若比较器只比较分数，分数相同的两个不同学生会被当作重复元素丢弃 → 比较器应补充唯一字段（如学号）。
- 对 HashSet 排序：转成 List 后 `sort`，或直接 `new TreeSet<>(set)`。

十一、Map

37. HashMap 底层结构（JDK 8）

- **数组 + 链表 + 红黑树**。数组每个位置称为桶（bucket），节点 `Node<K,V>` 含 hash、key、value、next。
- 默认初始容量 **16**，负载因子 **0.75**，阈值 threshold = 容量 × 负载因子；**容量始终为 2 的幂**；table 在**第一次 put 时才初始化**。
- **链表转红黑树**：链表长度 **≥ 8** 且数组容量 **≥ 64**；若容量 < 64 则**优先扩容**。红黑树节点数 **≤ 6** 时退化为链表（8 和 6 间留缓冲，避免频繁转换）。
- **JDK 7 vs 8**：7 为数组 + 链表、**头插法**、扩容需重新计算 hash；8 引入红黑树、**尾插法**、扩容按高位拆分。
- **key 可以为 null**：hash 值视为 0，放在下标 0 的桶；value 也可以为 null。

38. hash 计算与容量为 2 的幂

- 扰动函数：`hash = (h = key.hashCode()) ^ (h >>> 16)`，让高 16 位参与运算，减少低位相同导致的冲突。
- 下标：`index = (n - 1) & hash`，当 n 为 2 的幂时**等价于 `hash % n` 且位运算更快**。
- 2 的幂还使扩容时节点**要么留在原位置，要么移动到「原位置 + 旧容量」**，只需看 `hash & oldCap` 那一位。
- 传入非 2 的幂的初始容量时，`tableSizeFor` 会向上取整为 2 的幂（如 20 → 32）。

39. put 与 get 流程

- **put**：
  1. table 为空则 `resize()` 初始化；
  2. 计算 hash 和下标，桶为空直接放入新节点；
  3. 桶不为空：首节点 key 相同（hash 相等且 `==` 或 equals）则覆盖 value；是树节点则按红黑树插入；否则遍历链表，找到相同 key 覆盖，否则**尾插**，插入后链表长度达到 8 尝试树化；
  4. `++size > threshold` 则**扩容**。
- **get**：计算 hash 定位桶 → 检查首节点 → 红黑树查找 O(log n) 或链表遍历比较 hash 与 equals → 找不到返回 null。
- `get` 返回 null 无法区分「不存在」与「value 为 null」，应用 `containsKey` 判断。

40. 扩容机制

- 触发：元素个数 **size > threshold**（另外树化时容量 < 64 也会扩容）。
- 过程：新建 **2 倍**容量数组，遍历旧桶迁移；JDK 8 将链表按 `hash & oldCap` 拆分为**低位链（原下标）和高位链（原下标 + oldCap）**，保持原有顺序。
- 例：默认构造存 20 个元素 → 首次 put 初始化为 16（阈值 12）→ 第 13 个元素时扩容到 32（阈值 24），即**初始化 1 次 + 扩容 1 次**。
- `new HashMap<>(20)` → 实际容量 32，阈值 24，放 20 个元素不会再扩容；若想放 n 个不扩容，初始容量应设为 `n / 0.75 + 1`。
- **负载因子 0.75** 是空间利用率与哈希冲突概率的折中：太大冲突多，太小浪费空间、扩容频繁。

41. 为什么用红黑树而不是 AVL / 二叉搜索树

- 普通二叉搜索树可能退化为链表。
- **AVL 严格平衡**，查询略快，但插入删除需要更多旋转；**红黑树是弱平衡**，插入最多 2 次旋转、删除最多 3 次旋转，**增删效率更高**，整体性能更均衡。

- 链表长度达到 8 的概率在哈希分布良好时极低（源码注释中按泊松分布约为千万分之六），树化只是**应对极端冲突的兜底**。

42. HashMap 线程安全问题

- **JDK 7**：并发扩容时头插法导致链表**成环**，get 时死循环（CPU 100%）；也可能丢数据。
- **JDK 8**：尾插法解决了成环，但仍有**数据覆盖**（两个线程同时判断桶为空后写入）、`++size` 非原子导致 size 不准、扩容期间读到不完整数据等问题。
- 解决：`ConcurrentHashMap`（首选）、`Collections.synchronizedMap`、`Hashtable`。

43. 哈希冲突解决方法与 key 的选择

- 冲突解决：**链地址法**（HashMap 采用）、开放地址法（线性探测、二次探测，如 ThreadLocalMap 用线性探测）、再哈希法、建立公共溢出区。
- **String / Integer 适合做 key**：不可变（hash 不会变）、已正确重写 equals 和 hashCode、String 会缓存 hash 值。
- 自定义对象做 key：同时重写 equals 和 hashCode，且参与计算的字段不可变；equals 写错会导致相同 key 重复存储或取不到值。

44. Hashtable、LinkedHashMap、TreeMap

- **Hashtable**：方法级 `synchronized`，**key 和 value 都不允许 null**，默认容量 11、扩容为 2n+1，下标用取模；已被 ConcurrentHashMap 取代。
- **LinkedHashMap**：继承 HashMap，额外维护**双向链表**保证顺序：默认**插入顺序**；`accessOrder = true` 时为**访问顺序**。
  - **实现 LRU**：`new LinkedHashMap<>(cap, 0.75f, true)` + 重写 `removeEldestEntry` 在 size 超过容量时返回 true，自动淘汰最久未访问的头节点。
- **TreeMap**：基于**红黑树**，key 有序（自然顺序或 Comparator），操作 O(log n)；支持 `firstKey`、`floorKey`、`ceilingKey`、`subMap` 等范围操作；key 不能为 null（自然排序时）。适合需要**排序或范围查询**的场景。
- `keySet()`、`values()`、`entrySet()` 返回的是**视图**而非副本；遍历同时需要 key 和 value 时用 **`entrySet`**（或 `forEach`）效率最高。

45. ConcurrentHashMap 实现原理

- **JDK 7**：**Segment 分段锁**，Segment 继承 `ReentrantLock`（可重入），默认 16 个 Segment，即最多 16 个线程并发写；每个 Segment 内部是 HashEntry 数组 + 链表。
- **JDK 8**：取消 Segment，结构与 HashMap 相同（数组 + 链表 + 红黑树），**锁粒度细化到桶的首节点**：
  - **桶为空时用 CAS** 放入新节点（无锁）；
  - 桶非空时用 **synchronized 锁住首节点**再插入；

  - 发现首节点 hash 为 `MOVED`（ForwardingNode）说明正在扩容，当前线程**协助扩容**。
- **get 通常不加锁**：Node 的 `val` 和 `next` 用 **volatile** 修饰，table 也用 volatile + Unsafe 原子读，保证可见性。
- **为什么还要 CAS**：空桶插入是高频简单操作，CAS 乐观无锁性能更好；冲突时才用 synchronized（JDK 6 后锁优化使其性能足够好）。所以它是**乐观锁（CAS）+ 悲观锁（synchronized）结合**。

46. ConcurrentHashMap 扩容、size 与 null 限制

- **扩容**：多线程**协同迁移**，每个线程按步长（stride，最少 16 个桶）领取区间从后往前迁移；迁移完的桶放置 ForwardingNode，读操作遇到它会转到新表查找，写操作则先帮助扩容。`sizeCtl` 控制初始化与扩容状态。
- **size 统计**：类似 `LongAdder`，`baseCount` + `CounterCell[]` 数组分散计数，CAS 更新 baseCount 失败时改为更新某个 CounterCell；`size()` 求和，并发下只是**近似值**，不能作为精确判断依据。
- **不允许 null key / null value**：并发场景下 `get` 返回 null 时无法区分「不存在」还是「值为 null」，又无法像单线程那样用 containsKey 二次确认（两次调用之间可能被修改），存在**二义性**。
- **复合操作不安全**：先 `get` 判断再 `put` 不是原子的，应使用 **`putIfAbsent`**、**`computeIfAbsent`**、**`compute`**、**`merge`** 等原子方法。

47. HashMap、Hashtable、ConcurrentHashMap 对比

对比	HashMap	Hashtable	ConcurrentHashMap（JDK 8）
线程安全	否	是（方法级 synchronized）	是（CAS + 桶级 synchronized）
null	key、value 均可为 null	均不可	均不可
结构	数组 + 链表 + 红黑树	数组 + 链表	数组 + 链表 + 红黑树
性能	高	低（全表一把锁）	高并发下高
迭代器	fail-fast	fail-fast（Iterator）	弱一致

十二、Queue

48. Queue、Deque 与 PriorityQueue

- **Queue**（单端队列，FIFO）：两套方法——失败抛异常的 `add` / `remove` / `element`，与失败返回特殊值的 **`offer` / `poll` / `peek`**（推荐）。
- **Deque**（双端队列）：`offerFirst/offerLast`、`pollFirst/pollLast`，也可当栈用（`push/pop`）；**`ArrayDeque` 是首选的栈和队列实现**（比 `Stack`、`LinkedList` 更快），不允许 null。

- **PriorityQueue**：基于**数组实现的二叉堆，默认小顶堆**，可传 Comparator 改为大顶堆；入队/出队 O(log n)，取堆顶 O(1)；**用迭代器遍历不保证有序**，只有不断 `poll` 才按优先级输出；非线程安全，不允许 null。常用于 Top-K 问题。

49. BlockingQueue

- 阻塞队列：队列满时 `put` 阻塞，空时 `take` 阻塞；`offer/poll` 可带超时。内部基于 **ReentrantLock + Condition（notEmpty、notFull）** 实现生产者-消费者等待/唤醒。
- 常见实现：
  - **ArrayBlockingQueue**：数组、**有界**、一把锁；
  - **LinkedBlockingQueue**：链表、默认容量 `Integer.MAX_VALUE`（近似无界，**有 OOM 风险，应指定容量**）、**读写两把锁**，吞吐更高；
  - **SynchronousQueue**：不存储元素，每个 put 必须等待一个 take；
  - **PriorityBlockingQueue**：无界优先级队列；
  - **DelayQueue**：元素到期才能取出，用于延时任务。

第三部分 Go

十三、Go 基础

50. Go 语言优势与协程

- 优势：**语法简洁**、编译快、**静态编译单一二进制**部署方便、**原生并发（goroutine + channel）**、自带 GC、标准库强大（net/http）、工具链完善（fmt、test、pprof、race）。
- **进程**：资源分配的基本单位，独立地址空间；**线程**：CPU 调度的基本单位，共享进程资源，由内核调度，栈通常 MB 级；**协程（goroutine）**：用户态轻量级线程，由 **Go runtime 调度**。
- goroutine 优势：初始栈仅 **2KB**，按需增长（连续栈，扩容时复制到更大的栈）；创建/切换在用户态完成，开销远小于线程；可轻松创建数十万个。

51. make 与 new、数组与切片

- **new(T)**：分配 T 的零值内存，返回 **`*T` 指针**，可用于任何类型。
- **make(T, args)**：只用于 **slice、map、channel**，初始化其内部数据结构，返回 **T 本身**（非指针）。
- **数组**：长度是类型的一部分（`[3]int` 和 `[4]int` 是不同类型），**值类型**，赋值和传参会整体拷贝。
- **切片**：对底层数组的引用视图，长度可变，传参拷贝的是切片头（指针、len、cap）。

52. for range 的坑

- range 表达式只**求值一次**；遍历数组时遍历的是**数组副本**，遍历切片时复制的是切片头。
- **Go 1.22 之前**：循环变量在整个循环中**只有一个实例**，每次迭代覆盖其值，取地址 `&v` 得到的都是同一地址；闭包/goroutine 捕获循环变量会全部看到最后一个值。
- **Go 1.22 起**：`for` 循环的变量**每次迭代都是新变量**，上述问题消失（go.mod 中 go 版本 ≥ 1.22 生效）。
- 遍历 map 时修改 value 的字段需通过 `m[k] = v` 写回（map 元素不可寻址）。

53. 字符串、rune 与拼接

- string 底层是**只读字节数组 + 长度**，不可变；`len(s)` 是**字节数**。
- **byte = uint8**，表示一个字节；**rune = int32**，表示一个 Unicode 码点；`for _, r := range s` 按 rune 遍历（UTF-8 解码）；统计字符数用 `utf8.RuneCountInString` 或 `len([]rune(s))`。
- 拼接方式与性能：`+`（每次生成新串，少量拼接可用）、`fmt.Sprintf`（反射，较慢）、`strings.Join`、`bytes.Buffer`、**`strings.Builder`（推荐，可 `Grow` 预分配，`String()` 零拷贝）**。

- string 与 []byte 互转一般会发生**内存拷贝**（编译器对部分场景有优化）。

54. defer、panic 与 recover

- defer 按 **LIFO（后进先出）** 顺序执行，在函数 return 赋值返回值之后、真正返回之前执行。
- defer 的**参数在注册时就求值**；若用闭包则引用的是变量最终值。
- defer 可以**修改命名返回值**：`return x` 实际是「先给返回值赋值 → 执行 defer → 返回」。
- 场景：释放资源（关闭文件、解锁 `mu.Unlock()`）、recover 捕获 panic、记录耗时。
- **recover 只在 defer 函数中直接调用才有效**，且只能捕获**当前 goroutine** 的 panic；子 goroutine 的 panic 未 recover 会导致整个进程崩溃。
- 性能：Go 1.14 引入**开放编码（open-coded）defer**，常见场景下开销接近直接调用。

55. 空结构体、tag、init 与格式化

- **struct{} 不占内存**（`unsafe.Sizeof` 为 0，所有零大小对象可能指向同一地址 zerobase）。用途：
  - 实现集合 `map[string]struct{}`；
  - 只传信号的 channel `chan struct{}`；
  - 只有方法没有字段的类型。
- **struct tag**：字段元信息，如 `` `json:"name,omitempty"` ``，通过**反射** `reflect.StructTag.Get` 读取，用于 JSON、ORM、参数校验。
- **init 执行顺序**：先初始化依赖包 → 本包**包级变量** → 本包 **init()**（一个包可以有多个 init，按文件和出现顺序执行）→ `main()`。init 不能被显式调用。
- **打印**：`%v` 默认格式；`%+v` 结构体带**字段名**；`%#v` 输出 **Go 语法表示**（含类型名）；`%T` 输出类型。

56. 函数传参、多返回值与 nil 比较

- **Go 只有值传递**：slice、map、channel、指针传递的是「描述符/指针」的副本，所以函数内修改元素可影响外部，但重新赋值（如 append 导致扩容）不影响外部变量。
- **多返回值实现**：Go 1.17 之前通过**栈**传递参数和返回值；Go 1.17 起（amd64 等平台）采用**基于寄存器的调用约定**。
- **两个 nil 可能不相等**：interface 由（类型, 值）组成，只有**类型和值都为 nil** 时接口才等于 nil。
  - 例：`var p *T = nil; var i interface{} = p` → `i != nil`（类型为 *T）。函数返回自定义 error 时要直接 `return nil`，不要返回值为 nil 的具体类型指针。
- **两个 interface 可以比较**：动态类型相同且动态值相等时相等；若动态类型不可比较（slice、map、func）则**运行时 panic**。

57. unsafe.Pointer 与 uintptr

- 普通指针 `*T`：类型安全，不能进行指针运算，不同类型指针不能直接转换。
- **unsafe.Pointer**：通用指针，可与任意类型指针互转，**GC 会追踪**它指向的对象。
- **uintptr**：一个**整数**，可做算术运算，但 **GC 不把它当引用**，对象可能被回收或因栈增长被移动。
- 常见用法：`unsafe.Pointer` ↔ `uintptr` 做偏移计算后**在同一表达式内**转回 `unsafe.Pointer`，不要把 uintptr 存在变量中再转回。

十四、slice 与 map

58. slice 底层结构与扩容

- 结构 `runtime.slice`：`array`（指向底层数组的指针）、`len`、`cap`。
- **扩容规则（Go 1.18 起）**：
  1. 期望容量 > 2 倍旧容量 → 直接使用期望容量；
  2. 旧容量 **< 256** → 新容量翻倍；
  3. 否则循环 `newcap += (newcap + 3*256) / 4`，**从 2 倍平滑过渡到约 1.25 倍**；
  4. 最后按**内存规格（size class）向上取整**，所以实际 cap 可能比计算值略大。
- Go 1.18 之前：旧容量 < 1024 翻倍，否则每次增加 25%。
- 扩容会分配**新数组并拷贝**，原切片不受影响。

59. slice 共享底层数组的陷阱

- 切片截取 `s2 := s1[1:3]` 与原切片**共享底层数组**，修改 s2 元素会影响 s1。
- 对 s2 `append`：若未超过 cap，**会覆盖 s1 中对应位置的数据**；超过 cap 则扩容到新数组，此后两者互不影响。
- 可用**三索引切片** `s1[1:3:3]` 限制 cap，使 append 必然触发拷贝。
- 函数传 slice：修改元素影响外部；函数内 append 后外部变量的 len 不变，需要**返回新切片**或传 `*[]T`。
- 大数组截取小切片会导致整个大数组无法释放 → 用 `copy` 拷贝出需要的部分。

60. map 底层实现（经典 hmap 结构，Go 1.24 之前）

- `hmap`：`count`、`B`（桶数量为 2^B）、`buckets` 指针、`oldbuckets`（扩容时旧桶）、`hash0`（哈希种子）、`nevacuate`（迁移进度）。
- 桶 `bmap`：每个桶存 **8 个 key/value**，`tophash[8]` 存哈希值**高 8 位**用于快速比较；key 和 value 分别连续存放（减少内存对齐浪费）；满了通过 **overflow 溢出桶**链接。

- 查找：hash(key) → **低 B 位选桶** → 用**高 8 位**比较 tophash → 再比较完整 key → 桶内找不到则查溢出桶。
- **Go 1.24 起**内置 map 改为 **Swiss Table** 实现（以 8 个槽为一组，带控制字节，可用 SIMD/位运算批量匹配），面试仍以经典 hmap 结构为主，可补充说明。

61. map 扩容

- 触发时机（插入/修改时检查）：
  - **负载因子 > 6.5**（count / 2^B）→ **翻倍扩容**（B+1）；
  - **溢出桶过多**（B < 15 时溢出桶数 ⩾ 2^B；B ⩾ 15 时 ⩾ 2^15）→ **等量扩容**，重新整理使数据更紧凑（常见于大量删除后）。
- 过程：**渐进式迁移**——分配新桶，旧桶挂到 oldbuckets，之后每次**写入或删除**时迁移当前访问的桶并额外迁移一个桶，直到全部完成；翻倍扩容时旧桶数据按 hash 新增的那一位分流到 `原下标` 或 `原下标 + 旧桶数`。

62. map 使用要点

- **遍历无序**：runtime 每次遍历**随机选择起始桶和桶内起始位置**，有意避免开发者依赖顺序（扩容后顺序本来也会变）。需要有序：取出 key 排序后再按 key 访问。
- **并发不安全**：并发读写会触发 `fatal error: concurrent map read and map write`（无法 recover）。解决：`sync.RWMutex` 加锁、`sync.Map`、分片加锁 map。
- **key 必须可比较**：slice、map、func 不能做 key（需要 `==` 判断相等）；浮点数可以但 NaN 不等于自身，不建议使用。
- **不能对 map 元素取地址**：`&m[k]` 编译报错，扩容时元素会被迁移，地址不稳定。
- **删除 key 不会释放桶内存**：只标记槽位为空，map 不会缩容；大量删除后需重建新 map 才能释放内存。
- **可以边遍历边删除**：语言规范允许，被删除的未遍历元素不会再出现；遍历中新增的元素可能出现也可能不出现。

十五、channel 与 select

63. CSP 与 channel 底层结构

- **CSP（通信顺序进程）**：「**不要通过共享内存来通信，而要通过通信来共享内存**」，goroutine 之间通过 channel 传递数据。
- `hchan` 结构：
  - `buf`：**环形队列**（有缓冲 channel），`qcount` 元素个数、`dataqsiz` 容量、`sendx` / `recvx` 发送和接收下标；
  - `sendq` / `recvq`：阻塞的发送者/接收者**等待队列**（`sudog` 双向链表）；

    - `lock`：互斥锁，保护整个结构；
    - `closed`：关闭标志；`elemtype`、`elemsize`。
- channel 由 `makechan` 在**堆上**分配。

64. 发送与接收流程

- **发送 `ch <- v`**（加锁后）：
    1. `recvq` 有等待的接收者 → **直接把数据拷贝给它**并唤醒，不经过缓冲区；
    2. 缓冲区未满 → 数据拷贝到 `buf[sendx]`；
    3. 否则 → 当前 goroutine 封装为 sudog 加入 `sendq`，**gopark 挂起**，等待被接收者唤醒。
- **接收 `v := <-ch`**（加锁后）：
    1. `sendq` 有等待的发送者：无缓冲 → 直接从发送者拷贝；有缓冲（此时缓冲区必满）→ 取队头数据，并把发送者的数据放入队尾，唤醒发送者；
    2. 缓冲区有数据 → 从 `buf[recvx]` 取出；
    3. 否则 → 加入 `recvq` 挂起。
- 无缓冲 channel 是**同步**的：发送和接收必须同时就绪。

65. channel 操作结果总结

操作	nil channel	已关闭 channel	正常 channel
发送	永久阻塞	**panic**	阻塞或成功
接收	永久阻塞	先读完缓冲区剩余数据，之后返回**零值, false**	阻塞或成功
关闭	**panic**	**panic**	成功

- 关闭 channel 会唤醒所有等待的接收者（得到零值）和发送者（发送者会 panic）。
- 用 `v, ok := <-ch` 判断是否已关闭；`for v := range ch` 在关闭且读完后退出。
- **关闭原则**：由**发送方**关闭，**不要在接收端关闭**，多个发送者时通过额外的信号 channel 或 `sync.Once` 协调。

66. channel 导致的 goroutine 泄漏

- 场景：
    - 向无人接收的 channel 发送（如超时返回后，worker 仍向无缓冲 channel 发送结果）；
    - 从永远没有数据也不会关闭的 channel 接收；
    - `for range ch` 但发送方忘记 close；
    - 操作 nil channel。
- 预防：使用**带缓冲 channel**（容量 1 存结果）、`select` + `ctx.Done()` 退出、确保发送方关闭 channel；用 `runtime.NumGoroutine`、pprof `goroutine` 剖面或 `goleak` 排查。

67. select 机制与实现

- 每个 case 必须是 channel 操作；**多个 case 同时就绪时随机选择一个**，防止饥饿；都不就绪时有 `default` 走 default（非阻塞），否则阻塞。
- 特例：空 `select {}` **永久阻塞**；只有一个 case 编译器改写为普通收发；一个 case + default 改写为非阻塞收发（`selectnbsend` / `selectnbrecv`）。
- 运行时 `runtime.selectgo`：
  1. 生成**随机轮询顺序 pollorder** 和按 channel 地址排序的**加锁顺序 lockorder**（避免死锁）；
  2. 按 lockorder 锁定所有 channel，按 pollorder 查找就绪 case，找到则执行并返回；
  3. 无就绪且无 default → 当前 goroutine 的 sudog 加入**所有** channel 的等待队列并挂起；
  4. 被某个 channel 唤醒后，从其他 channel 的等待队列中移除自己。
- 常用：`select` + `time.After` 实现超时；`for-select` + `ctx.Done()` 实现可取消循环（循环中频繁使用 `time.After` 应改为复用 `time.Timer`）。

十六、sync 与并发原语

68. 安全读写共享变量的方式

- **sync.Mutex / RWMutex** 互斥锁、读写锁；
- **channel** 串行化访问（由单个 goroutine 持有数据）；
- **sync/atomic** 原子操作（`atomic.Int64`、`atomic.Value`、CAS）；
- `sync.Once`、`sync.Map`、`sync.Pool` 等；
- 并发安全检测：`go test -race` / `go run -race`。

69. 原子操作与锁

- 原子操作由 **CPU 指令**保证（x86 上如 `LOCK` 前缀指令、`XCHG`、`CMPXCHG`），不可被中断。
- 区别：
  - 原子操作针对**单个变量**，无上下文切换，性能高，属于乐观/无锁思想（CAS 失败重试）；
  - 锁可保护**一段临界区**的多个操作，竞争激烈时 goroutine 会休眠，属于悲观思想。
- Go 1.19 起提供 `atomic.Int32`、`atomic.Int64`、`atomic.Bool`、`atomic.Pointer[T]` 等类型化原子类型。

70. Mutex 实现与两种模式

- 结构：`state int32`（低位依次为 **locked、woken、starving 标志，其余位为等待者数量**）+ `sema` 信号量。
- 加锁：快速路径 **CAS** 将 state 从 0 改为 locked；失败进入慢路径，满足条件时**自旋**（多核、GOMAXPROCS > 1、有其他运行中的 P 且本地队列为空、自旋次数 < 4），否则通过信号量休

眠排队。

- **正常模式**：等待者按 FIFO 排队，但被唤醒的 goroutine 要与**新到来的 goroutine 竞争**锁，新来者在 CPU 上运行、更容易抢到 → 吞吐高但可能让队列中的等待者饥饿。
- **饥饿模式**：等待者等待**超过 1ms** 时切换；锁**直接交给队头等待者**，新来的 goroutine 不自旋、不抢锁，直接排到队尾。当获得锁的等待者是队列最后一个或等待时间 < 1ms 时切回正常模式。
- 自旋时间很短且有次数限制，不会长期占用 CPU。
- Mutex **不可重入**，同一 goroutine 重复加锁会死锁；不要复制已使用的 Mutex（`go vet` 可检查）。
- **RWMutex**：读锁共享、写锁独占；有写锁等待时新的读锁会阻塞，**防止写饥饿**；适合读多写少。

71. sync.Once 与 WaitGroup

- **sync.Once**：保证函数只执行一次（单例、懒加载配置）。
  - 结构：`done`（原子标志）+ `m Mutex`；
  - 快速路径**原子读** done == 1 直接返回；慢路径加锁后**双重检查**，执行 f 后再将 done 原子置 1（f 执行完才置位，保证其他 goroutine 返回时 f 已完成）；
  - f 中 panic 也视为已执行；f 内再调用同一个 Once 的 Do 会死锁。Go 1.21 新增 `sync.OnceFunc`、`OnceValue`。
- **sync.WaitGroup**：等待一组 goroutine 完成。
  - 内部一个原子状态：**计数器（高 32 位）+ 等待者数量（低 32 位）**，配合信号量；
  - `Add(n)` 增加计数（应在启动 goroutine **之前**调用），`Done()` 即 `Add(-1)`，计数为 0 时唤醒所有 `Wait` 的 goroutine；计数变为负数会 panic；
  - Go 1.25 新增 `wg.Go(func)`，自动完成 Add 与 Done。

72. sync.Map

- 适用场景（官方说明）：**key 只写一次读多次（只增长的缓存）**，或**多个 goroutine 读写互不相交的 key 集合**。写多、key 频繁变化时不如 `map + RWMutex`。
- 经典实现（Go 1.24 之前）：**read map（只读，原子访问，无锁）+ dirty map（加锁访问）+ misses 计数**。
  - **Load**：先查 read；未命中且 `amended=true`（dirty 中有 read 没有的 key）时加锁查 dirty，misses++；**misses ≥ len(dirty) 时把 dirty 提升为 read**；
  - **Store**：key 在 read 中且未被标记 expunged → 直接 CAS 更新 entry；否则加锁写入 dirty（dirty 为 nil 时先从 read 复制未删除的条目）；
  - **Delete**：read 中存在则把 entry 置为 nil（**延迟删除**）。

- **nil 与 expunged 两种删除状态**：nil 表示已删除但 dirty 中可能仍有该 key；expunged 表示已删除**且 dirty 中没有**，复制 read → dirty 时把 nil 标记为 expunged 并跳过，之后对该 key 的写入必须加锁同时写入 dirty，保证 dirty 提升为 read 时数据完整。
- Go 1.24 起 sync.Map 底层改为并发哈希字典树（HashTrieMap）实现，API 不变。

十七、context

73. context 的作用与接口

- 作用：在 goroutine 调用链之间传递**取消信号、超时/截止时间、请求域数据**，典型用于 HTTP 请求、RPC、数据库调用的超时控制与级联取消。
- `Context` 接口四个方法：`Deadline()`、`Done()`（返回只读 channel，取消时关闭）、`Err()`（`Canceled` 或 `DeadlineExceeded`）、`Value(key)`。
- 创建：根 `context.Background()` / `context.TODO()`；派生 `WithCancel`、`WithTimeout`、`WithDeadline`、`WithValue`；Go 1.20 `WithCancelCause`，Go 1.21 `WithoutCancel`、`AfterFunc`。

74. 取消传播与 Value 查找

- **树形结构**：父 context 取消时，会**递归取消所有子 context**（cancelCtx 维护 children 集合）；子取消不影响父。
- 取消过程：关闭 `done` channel、设置 err、遍历取消子节点、从父节点的 children 中移除自己。`WithTimeout` 基于 `time.AfterFunc` 定时器到期自动调用 cancel。
- **Value 查找**：valueCtx 每个节点只存一个 k/v，查找时**沿父链向上逐层查找**，O(n)，找不到返回 nil。
- 最佳实践：
  - 作为函数**第一个参数** `ctx context.Context`，不要放进结构体；
  - **WithCancel/WithTimeout 返回的 cancel 必须调用**（`defer cancel()`），否则定时器和子节点资源泄漏；
  - Value 只传**请求域数据**（traceID、用户信息），key 使用自定义私有类型避免冲突，不要传可选业务参数；
  - 不要传 nil context，不确定时用 `context.TODO()`。

十八、interface 与反射

75. interface 底层：eface 与 iface

- **eface**（空接口 `interface{}` / `any`）：`_type`（动态类型信息）+ `data`（指向数据的指针）。
- **iface**（带方法的接口）：`tab *itab` + `data`。`itab` 包含**接口类型、动态类型、方法表 fun**（按方法名排序的函数指针），itab 会被全局缓存。

- **隐式实现（鸭子类型）**：类型实现了接口的全部方法就自动实现该接口，可用 `var _ I = (*T)(nil)` 在编译期校验。
- **方法集规则**：值类型 T 的方法集只含值接收者方法；指针 `*T` 的方法集包含值和指针接收者方法 → 用指针接收者实现接口时，只有 `*T` 实现了该接口。

76. 类型转换与类型断言

- **类型转换** `T(x)`：编译期在兼容类型间转换（如 `int64(i)`、`[]byte(s)`）。
- **类型断言** `x.(T)`：只能用于**接口值**，运行时检查动态类型；`v, ok := x.(T)` 失败时 ok 为 false，单返回值形式失败会 panic。
- **type switch**：`switch v := x.(type) { case int: ... }`。
- interface 应用：依赖抽象便于 mock 测试、标准库 `io.Reader/Writer`、`error`、`sort.Interface`、多态与插件化。

77. 反射

- 基于 interface 的类型信息，在**运行时**获取和修改变量的类型与值：`reflect.TypeOf()` 返回 `reflect.Type`，`reflect.ValueOf()` 返回 `reflect.Value`。
- **反射三大定律**：接口值 → 反射对象；反射对象 → 接口值（`v.Interface()`）；**要修改反射对象，值必须可设置**（传指针并用 `Elem()`，`CanSet()` 为 true）。
- 应用：`encoding/json` 序列化、ORM 映射、struct tag 解析、依赖注入、`fmt` 打印。
- 缺点：性能较差、失去编译期类型检查、代码可读性差。
- 比较两个对象完全相同：`reflect.DeepEqual`（递归比较，可比较 slice、map）；可比较类型直接用 `==`。

十九、GMP 调度模型

78. GMP 模型

- **G（Goroutine）**：协程，包含栈、程序计数器、状态等。
- **M（Machine）**：**操作系统线程**，真正执行代码，必须绑定 P 才能执行 G（默认最多 10000 个，可用 `debug.SetMaxThreads` 调整）。
- **P（Processor）**：逻辑处理器，持有**本地运行队列**（最多 **256** 个 G）和 `runnext`（下一个优先运行的 G）、mcache 等资源；数量由 **GOMAXPROCS** 决定（默认等于 CPU 核数；Go 1.25 起在容器中会考虑 cgroup CPU 限制）。
- **全局运行队列**：本地队列满时，一半 G 被移到全局队列，访问全局队列需要加锁。
- 调度器（scheduler）作用：把大量 G 高效地分配到少量 M 上执行，实现 M:N 调度。

79. 调度策略

- **work stealing（工作窃取）**：P 的本地队列为空时，先查全局队列，再从其他 P 的本地队列**偷一半** G。
- **hand off（移交）**：G 执行**阻塞系统调用**时，M 与 P 解绑，P 交给其他空闲 M（或新建 M）继续运行其他 G；系统调用返回后 G 尝试获取空闲 P，获取不到则放入全局队列。
- **抢占**：
  - Go 1.14 之前：**基于协作的抢占**，在函数调用时检查抢占标志，纯计算死循环无法被抢占；
  - **Go 1.14 起：基于信号的异步抢占**，sysmon 发现 G 运行超过 **10ms**，向其所在 M 发送 **SIGURG** 信号强制抢占。
- **sysmon 监控线程**：不需要绑定 P，负责抢占长时间运行的 G、回收系统调用中阻塞的 P、netpoll 检查、触发强制 GC。

80. 调度时机与查找可运行 G 的顺序

- 调度时机：
  - 主动让出：`runtime.Gosched()`；
  - **阻塞**：channel 收发、锁、`time.Sleep`、网络 IO（netpoller）等调用 `gopark`；
  - 被抢占：运行超过 10ms；
  - 系统调用；G 执行结束。
- M 寻找可运行 G（`findRunnable`）：
  1. 每调度 **61 次**优先查一次**全局队列**（防止全局队列饥饿）；
  2. 本地队列（先 `runnext`）；
  3. 全局队列（批量取一部分）；
  4. **netpoll** 获取 IO 就绪的 G；
  5. 从其他 P **窃取**；
  6. 都没有则 M 休眠。

81. 为什么需要 P、g0 与 m0

- **早期 GM 模型（Go 1.1 前）的问题**：所有 G 放在**全局队列，需全局锁**，竞争激烈；G 在不同 M 间频繁切换，局部性差；每个 M 都持有内存缓存，浪费内存；系统调用阻塞时线程切换频繁。
- 引入 P 后：**本地队列无锁访问**、内存缓存 mcache 挂在 P 上（数量 = GOMAXPROCS），并支持 work stealing 和 hand off。
- P 在程序启动时按 GOMAXPROCS 创建；M 在没有足够的 M 关联 P 运行可运行 G 时创建（如系统调用阻塞时）。

- **m0**：进程启动的**第一个线程**，负责初始化运行时和启动第一个 G（main goroutine），之后与普通 M 一样。
- **g0**：每个 M 都有一个 g0，**专门负责调度**（执行 schedule、创建 G、栈扩容、GC 辅助等运行时代码），使用固定大小的系统栈而非可增长的协程栈；用户 G 需要调度时先切换到 g0 栈（`mcall` / `systemstack`），选出下一个 G 后再切换到其用户栈（`gogo`）。

二十、内存管理与 GC

82. 内存分配

- 借鉴 **TCMalloc**，多级缓存：
  - **mcache**：每个 P 一个，**无锁**分配；
  - **mcentral**：每种规格一个，mcache 不足时从这里加锁获取 span；
  - **mheap**：全局堆，管理页（8KB），向操作系统申请内存。
- **mspan**：由连续页组成，按对象大小分为约 **68 种规格（size class）**。
- 按对象大小分配：
  - **微对象**（< 16B 且不含指针）：tiny 分配器合并到同一 16B 块；
  - **小对象**（16B ~ 32KB）：按 size class 从 mcache 对应 span 分配；
  - **大对象**（> 32KB）：直接从 mheap 分配。

83. 内存逃逸

- **逃逸分析**：编译器决定变量分配在**栈**（函数返回自动回收，开销小）还是**堆**（由 GC 回收）。查看：`go build -gcflags="-m"`。
- 常见逃逸场景：
  - 函数**返回局部变量的指针**；
  - 变量赋值给 **interface 类型**（如传给 `fmt.Println` 的参数）；
  - **闭包**引用外部变量；
  - **栈空间不足**/对象过大，或切片长度在运行时才确定（`make([]int, n)`）；
  - 向 channel 发送指针、指针存入 slice/map。
- 影响：堆分配更慢，并**增加 GC 压力**。优化：小对象尽量值传递、预分配容量、避免不必要的 interface 和指针、使用 `sync.Pool` 复用对象。

84. 内存泄漏场景与排查

- 常见原因（Go 有 GC 仍会泄漏，因为对象**仍被引用**）：
  - **goroutine 泄漏**（最常见）：channel 阻塞、无退出条件的循环、锁未释放；
  - 子切片/子字符串引用大数组；

    - 全局 map / 缓存只增不删；
    - `time.Ticker` 未 Stop（Go 1.23 起未引用的 Timer/Ticker 可被 GC 回收）；
    - 未关闭的 HTTP response body、文件等资源。
- 排查：引入 `net/http/pprof`，`go tool pprof` 分析 **heap**（内存）和 **goroutine** 剖面，对比两个时间点的差异；监控 `runtime.NumGoroutine()`；测试中使用 `go.uber.org/goleak`。

85. 三色标记法与 GC 根对象

- Go GC：**并发三色标记清除**，非分代、非整理（不移动对象）。
- 三色：
    - **白色**：未被访问，标记结束后仍为白色的对象会被回收；
    - **灰色**：已被访问，但其引用的对象还未扫描完；
    - **黑色**：自身及引用的对象都已扫描。
- 过程：根对象置灰 → 不断从灰色集合取出对象，将其引用对象置灰、自身置黑 → 灰色集合为空时回收所有白色对象。
- **根对象**：全局变量（数据段、BSS 段）、每个 goroutine **栈上的变量**、寄存器中的指针。
- **STW（Stop The World）**：暂停所有用户 goroutine，Go 的 STW 通常在亚毫秒级，只发生在标记开始和标记终止阶段。

86. 写屏障与混合写屏障

- 并发标记的难点：用户程序同时修改引用，可能导致**存活对象被误回收**。发生条件（**三色不变式被破坏**）：
    1. 黑色对象引用了白色对象；
    2. 同时灰色对象到该白色对象的路径被删除。
- **强三色不变式**：黑色对象不能直接引用白色对象 → **插入写屏障**（Dijkstra）：新增引用时把被引用对象置灰。缺点：栈上不开启写屏障，标记结束需 **STW 重新扫描栈**。
- **弱三色不变式**：黑色引用的白色对象必须有灰色对象可达保护 → **删除写屏障**（Yuasa）：删除引用时把被删除的对象置灰。缺点：回收精度低，需要起始时快照栈。
- **混合写屏障（Go 1.8）**：结合两者——
    - GC 开始时**将栈上可达对象全部标记为黑色**（逐个 goroutine 扫描，无需整体 STW，之后**无需重新扫描栈**）；
    - GC 期间栈上**新创建的对象为黑色**；
    - 堆上**被删除的引用对象和被添加的引用对象都标记为灰色**。
    - 大幅缩短 STW 时间。

87. GC 流程、触发时机与调优

- **流程**：

1. **清扫终止**（STW）：完成上一轮清扫，准备开启写屏障；
2. **并发标记**：开启写屏障，后台标记 worker 约占用 **25% CPU**，分配内存过快的 goroutine 需进行**辅助标记（mark assist）**；
3. **标记终止**（STW）：关闭写屏障，计算下次触发目标；
4. **并发清扫**：回收白色对象，可在分配内存时惰性清扫。

- **触发时机**：
  - 堆内存达到目标值：**GOGC（默认 100）**表示新分配内存达到上次 GC 后存活堆的 100% 时触发；
  - **定时触发**：超过 **2 分钟**未 GC 时 sysmon 强制触发；
  - 手动调用 `runtime.GC()`；
  - 接近 **GOMEMLIMIT**（Go 1.19 引入的软内存上限）时会更积极地 GC。
- **关注指标**：CPU 占用率、STW 停顿时间、GC 频率、堆内存峰值。
- **调优**：
  - **减少堆分配**（最有效）：预分配 slice/map 容量、`sync.Pool` 复用对象、避免逃逸、`strings.Builder` 拼接；
  - 调整 **GOGC**（调大降低 GC 频率但占用更多内存）；
  - 设置 **GOMEMLIMIT** 防止 OOM，容器环境下常配合使用。
- **观察**：`GODEBUG=gctrace=1`、`runtime.ReadMemStats`、pprof（heap / allocs）、`go tool trace`。
- 补充：Go 1.25 以实验特性（`GOEXPERIMENT=greenteagc`）引入新的「Green Tea」GC，优化小对象标记的内存局部性。

二十一、常见并发模式与代码题

88. 常见并发模式

- **Worker Pool**：固定数量 goroutine 从任务 channel 取任务，控制并发度。
- **Pipeline 流水线**：每个阶段一个 goroutine，通过 channel 串联，上游关闭 channel 通知下游。
- **Fan-out / Fan-in**：多个 goroutine 并行处理同一输入（扇出），再合并到一个 channel（扇入）。
- **errgroup**（`golang.org/x/sync/errgroup`）：并发执行一组任务，任一出错可通过 ctx 取消其余任务，`SetLimit` 限制并发数。
- **singleflight**：合并相同 key 的并发请求，防缓存击穿。
- **限流**：带缓冲 channel 作信号量；`golang.org/x/time/rate` 令牌桶；`time.Ticker` 匀速。
- **超时控制**：`context.WithTimeout` / `select` + `time.After`。
- **优雅退出**：监听 `os.Signal`，`http.Server.Shutdown(ctx)`，`WaitGroup` 等待在途任务结束。

89. 并发控制 + 超时（代码模板）

- 需求：1000 个任务，最多 10 个并发，整体超时 1 秒。

```
func run(tasks []func(ctx context.Context) error) error {
    ctx, cancel := context.WithTimeout(context.Background(), time.Second)
    defer cancel()

    g, ctx := errgroup.WithContext(ctx)
    g.SetLimit(10) // 最多 10 个 goroutine 同时运行
    for _, t := range tasks {
        g.Go(func() error { return t(ctx) }) // Go 1.22+ 循环变量每次迭代独立
    }
    return g.Wait() // 任一任务出错或超时，ctx 被取消
}
```

- 不依赖第三方库时：`sem := make(chan struct{}, 10)`，启动前 `sem <- struct{}{}`、结束时 `<-sem`，配合 `WaitGroup` 与 `select { case <-ctx.Done(): }`。

90. 交替打印（代码模板）

- 需求：两个 goroutine 交替打印奇偶数 1~10（思路可推广到三个 goroutine 循环打印 abc：用 N 个 channel 形成环）。

```
func main() {
    odd, even := make(chan struct{}), make(chan struct{})
    var wg sync.WaitGroup
    wg.Add(2)
    go func() { // 打印奇数
        defer wg.Done()
        for i := 1; i <= 10; i += 2 {
            <-odd
            fmt.Println(i)
            even <- struct{}{}
        }
    }()
    go func() { // 打印偶数
        defer wg.Done()
        for i := 2; i <= 10; i += 2 {
            <-even
            fmt.Println(i)
            if i < 10 {
                odd <- struct{}{}
            }
        }
    }()
    odd <- struct{}{} // 启动信号
    wg.Wait()
}
```

- 要点：用**无缓冲 channel 传递「令牌」**保证顺序；注意最后一轮不要再发送令牌，否则会因无人接收而死锁。也可用 `sync.Cond` 或 `sync.Mutex` + 共享计数实现。